JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ServerSocketChannel

java.lang.Object
  java.nio.channels.spi.AbstractInterruptibleChannel
    java.nio.channels.SelectableChannel
      java.nio.channels.spi.AbstractSelectableChannel
        java.nio.channels.ServerSocketChannel

All Implemented Interfaces:

`Closeable`, `AutoCloseable`, `Channel`, `InterruptibleChannel`, `NetworkChannel`

```
public abstract class ServerSocketChannel
extends AbstractSelectableChannel
implements NetworkChannel
```

A selectable channel for stream-oriented listening sockets.

A server-socket channel is created by invoking one of the `open` methods of this class. The no-arg `open` method opens a server-socket channel for an *Internet protocol* socket. The `open(ProtocolFamily)` method is used to open a server-socket channel for a socket of a specified protocol family. It is not possible to create a channel for an arbitrary, pre-existing socket. A newly-created server-socket channel is open but not yet bound. An attempt to invoke the `accept` method of an unbound server-socket channel will cause a `NotYetBoundException` to be thrown. A server-socket channel can be bound by invoking one of the `bind` methods defined by this class.

Socket options are configured using the `setOption` method. Server-socket channels for *Internet protocol* sockets support the following options:

Option Name	Description
`SO_RCVBUF`	The size of the socket receive buffer
`SO_REUSEADDR`	Re-use address

Server-socket channels for *Unix domain* sockets support:

Option Name	Description
`SO_RCVBUF`	The size of the socket receive buffer

Additional (implementation specific) options may also be supported.

Server-socket channels are safe for use by multiple concurrent threads.

Since:

1.4

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	ServerSocketChannel (SelectorProvider provid	Initializes a new instance of this class.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
abstract SocketChannel	accept()	Accepts a connection made to this channel's socket.
final ServerSocketChannel	bind (SocketAddress local)	Binds the channel's socket to a local address and configures the socket to listen for connections.
abstract ServerSocketChannel	bind (SocketAddress local, int backlog)	Binds the channel's socket to a local address and configures the socket to listen for connections.
abstract SocketAddress	getLocalAddress()	Returns the socket address that this channel's socket is bound to.
static ServerSocketChannel	open()	Opens a server-socket channel for an *Internet protocol* socket.
static ServerSocketChannel	open (ProtocolFamily family)	Opens a server-socket channel.
abstract <T> ServerSocketChan		Sets the value of a socket option.

	setOption(SocketOption<T> name, T value)	
abstract ServerSocket	**socket**()	Retrieves a server socket associated with this channel.
final int	**validOps**()	Returns an operation set identifying this channel's supported operations.

Methods declared in class AbstractSelectableChannel

blockingLock, configureBlocking, implCloseChannel, implCloseSelectableChannel, implConfigureBlocking, isBlocking, isRegistered, keyFor, provider, register

Methods declared in class SelectableChannel

register

Methods declared in class AbstractInterruptibleChannel

begin, close, end, isOpen

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Methods declared in interface NetworkChannel

getOption, supportedOptions

Constructor Details

ServerSocketChannel

```
protected ServerSocketChannel(SelectorProvider provider)
```

Initializes a new instance of this class.

Parameters:

`provider` - The provider that created this channel

Method Details

open

```
public static ServerSocketChannel open()
                                  throws IOException
```

Opens a server-socket channel for an *Internet protocol* socket.

The new channel is created by invoking the `openServerSocketChannel` method of the system-wide default `SelectorProvider` object.

The new channel's socket is initially unbound; it must be bound to a specific address via one of its socket's `bind` methods before connections can be accepted.

Returns:

A new socket channel

Throws:

`IOException` - If an I/O error occurs

See Also:

java.net.preferIPv4Stack system property

open

```
public static ServerSocketChannel open(ProtocolFamily family)
                                  throws IOException
```

Opens a server-socket channel. The `family` parameter specifies the `protocol family` of the channel's socket.

The new channel is created by invoking the `openServerSocketChannel(ProtocolFamily)` method of the system-wide default `SelectorProvider` object.

Parameters:

`family` - The protocol family

Returns:

A new socket channel

Throws:

`UnsupportedOperationException` - If the specified protocol family is not supported. For example, suppose the parameter is specified as `StandardProtocolFamily.INET6` but IPv6 is not enabled on the platform.

`IOException` - If an I/O error occurs

Since:

15

See Also:

java.net.preferIPv4Stack system property

validOps

```
public final int validOps()
```

Returns an operation set identifying this channel's supported operations.

Server-socket channels only support the accepting of new connections, so this method returns `SelectionKey.OP_ACCEPT`.

Specified by:

`validOps` in class `SelectableChannel`

Returns:

The valid-operation set

bind

```
public final ServerSocketChannel bind(SocketAddress local)
                                 throws IOException
```

Binds the channel's socket to a local address and configures the socket to listen for connections.

An invocation of this method is equivalent to the following:

```
    bind(local, 0);
```

Specified by:

`bind` in interface `NetworkChannel`

Parameters:

`local` - The local address to bind the socket, or `null` to bind to an automatically assigned socket address

Returns:

This channel

Throws:

`AlreadyBoundException` - If the socket is already bound

`UnsupportedAddressTypeException` - If the type of the given address is not supported

`ClosedChannelException` - If the channel is closed

`IOException` - If some other I/O error occurs

Since:

1.7

See Also:

`NetworkChannel.getLocalAddress()`

bind

```
public abstract ServerSocketChannel bind(SocketAddress local,
                                         int backlog)
                                  throws IOException
```

Binds the channel's socket to a local address and configures the socket to listen for connections.

This method is used to establish an association between the socket and a local address. For *Internet protocol* sockets, once an association is established then the socket remains bound until the channel is closed.

The `backlog` parameter is the maximum number of pending connections on the socket. Its exact semantics are implementation specific. In particular, an implementation may impose a maximum length or may choose to ignore the parameter altogether. If the `backlog` parameter has the value `0`, or a negative value, then an implementation specific default is used.

API Note:

Binding a server socket channel for a *Unix Domain* socket, creates a file corresponding to the file path in the `UnixDomainSocketAddress`. This file persists after the channel is closed, and must be removed before another socket can bind to the same name. Binding to a `null` address causes the socket to be *automatically* bound to some unique file in a system temporary location. The associated socket file also persists after the channel is closed. Its name can be obtained from the channel's local socket address.

Implementation Note:

Each platform enforces an implementation specific, maximum length for the name of a *Unix Domain* socket. This limitation is enforced when a channel is bound. The maximum length is typically close to and generally not less than 100 bytes. This limitation also applies to *automatically* bound server socket channels. See the *Unix domain* networking properties that can be used to select the temporary directory where these sockets are created.

Parameters:

`local` - The address to bind the socket, or `null` to bind to an automatically assigned socket address

`backlog` - The maximum number of pending connections

Returns:

This channel

Throws:

`AlreadyBoundException` - If the socket is already bound

`UnsupportedAddressTypeException` - If the type of the given address is not supported

`ClosedChannelException` - If this channel is closed

`IOException` - If some other I/O error occurs

Since:

1.7

setOption

```
public abstract <T> ServerSocketChannel setOption
(SocketOption<T> name,
 T value)
                                                throws IOException
```

Description copied from interface: `NetworkChannel`

Sets the value of a socket option.

Specified by:

`setOption` in interface `NetworkChannel`

Type Parameters:

`T` - The type of the socket option value

Parameters:

`name` - The socket option

`value` - The value of the socket option. A value of `null` may be a valid value for some socket options.

Returns:

This channel

Throws:

`UnsupportedOperationException` - If the socket option is not supported by this channel

`IllegalArgumentException` - If the value is not a valid value for this socket option

`ClosedChannelException` - If this channel is closed

`IOException` - If an I/O error occurs

Since:

1.7

See Also:

StandardSocketOptions

socket

```
public abstract ServerSocket socket()
```

Retrieves a server socket associated with this channel.

The returned object will not declare any public methods that are not declared in the ServerSocket class.

Returns:

A server socket associated with this channel

Throws:

UnsupportedOperationException - If the channel's socket is not an *Internet protocol* socket

accept

```
public abstract SocketChannel accept()
                              throws IOException
```

Accepts a connection made to this channel's socket.

If this channel is in non-blocking mode then this method will immediately return `null` if there are no pending connections. Otherwise it will block indefinitely until a new connection is available or an I/O error occurs.

The socket channel returned by this method, if any, will be in blocking mode regardless of the blocking mode of this channel.

Returns:

The socket channel for the new connection, or `null` if this channel is in non-blocking mode and no connection is available to be accepted

Throws:

ClosedChannelException - If this channel is closed

AsynchronousCloseException - If another thread closes this channel while the accept operation is in progress

ClosedByInterruptException - If another thread interrupts the current thread while the accept operation is in progress, thereby closing the channel and setting the current thread's interrupted status

NotYetBoundException - If this channel's socket has not yet been bound

IOException - If some other I/O error occurs

getLocalAddress

```
public abstract SocketAddress getLocalAddress()
                                        throws IOException
```

Returns the socket address that this channel's socket is bound to.

Where the channel is bound to an Internet Protocol socket address then the return value from this method is of type InetSocketAddress.

Where the channel is bound to a *Unix Domain* socket address, the socket address is a UnixDomainSocketAddress.

Specified by:

getLocalAddress in interface NetworkChannel

Returns:

The SocketAddress that the socket is bound to or null if the channel's socket is not bound

Throws:

ClosedChannelException - If the channel is closed

IOException - If an I/O error occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.